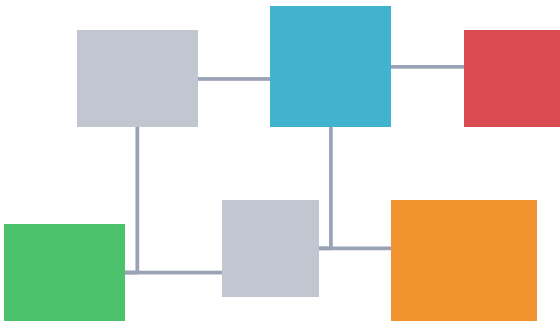


Hybrid Dungeon Kit

Procedural 2D dungeon generator for Unity

User Manual · Version 1.0



A procedural 2D dungeon generator combining BSP partitioning with WFC-inspired room constraints. Every feature is driven from the Inspector or from code.

Contents

1. Installation	3
2. Quick start (30 seconds)	4
3. The generator Inspector	5
4. Tiles & themes	6
Rule Tiles and animated tiles	6
Switching themes	6
5. Boss, treasure & shop rooms	7
Boss rooms	7
Treasure and shop rooms	7
6. Multi-level runs (optional)	8
7. Rooms, spawning & events	9
Solid walls	9
Room data	9
Spawning enemies and loot	9
Reacting to the player's position	10
8. Minimap & fog of war	11
Minimap	11
Fog of war	11
9. Saving, loading & baking	12
Save and load layouts (JSON)	12
Baking to a prefab	12
10. Scripting API	13
11. Samples	14
12. Performance & async generation	15
13. FAQ & troubleshooting	16
14. Support	17

1. Installation

- 1 Import the package: **Assets → Import Package → Custom Package...** and select `HybridDungeonKit.unitypackage` (keep everything checked).
- 2 Requirements: Unity 6 (6000.0) or newer; developed and tested on Unity 6.3. Works with the Built-in Render Pipeline and URP. Needs the **2D Tilemap** package, which is included by default in 2D projects.

The kit lives entirely in one folder (`Assets/HybridDungeonKit/`) and uses its own assembly definitions and the `HybridDungeonKit` namespace, so it never clashes with your own code.

2. Quick start (30 seconds)

See everything in action: open `HybridDungeonKit/Examples/HybridDungeonKit_Demo` and press Play. Walk with WASD / arrows (or a gamepad); press **R** for a new dungeon, **T** to switch theme, **M** to toggle the minimap, **F** to reveal the map. If the scene is missing, create it with **Tools → Hybrid Dungeon Kit → Create Playable Demo Scene** (run **Build Sample Theme** first for the pixel-art tiles).

Add a generator to your own scene:

- 1 Menu → **Tools → Hybrid Dungeon Kit → Create Dungeon Generator** (or **Create Demo Scene** for a ready-made scene). This creates a generator already wired to a Grid with Floor / Wall / Corridor tilemaps.
- 2 Select the generator and click **Generate** in the Inspector.
- 3 A dungeon is drawn immediately in the Scene view — no Play mode, and no tiles required. Placeholder colour tiles are used until you assign your own art.

3. The generator Inspector

Action buttons

- **Initialize Scene** — create or link a Grid + Floor/Wall/Corridor tilemaps and wire them to this generator.
- **Generate** — build a dungeon with the current settings.
- **Clear** — remove the drawn tiles and any spawned boss objects.
- **Regenerate (new seed)** — force a fresh seed and rebuild.
- **Bake to Prefab** — save the current dungeon as a standalone prefab (see 9).
- **Save Layout... / Load Layout...** — save the current dungeon as JSON, or load one back (see 9).
- **Quick size presets** — Small / Medium / Large one-click sizing.

Settings sections (collapsible, ordered most-used first)

- **Essentials** — Layout Mode (see below), Target Room Count, grid Width/Height, Boss / Treasure / Shop Rooms Per Dungeon, Use Random Seed, Seed. Most dungeons only need these.
- **Rooms & Corridors** — Corridor Width (1–3 cells), Min/Max Room Size, Max Corridors Per Room, and the Rectangle / Opposed-Bump / Opposed-Notch shape weights.
- **Markers & Setup** — Boss Prefab, Mark Boss Tiles, Mark Special Room Tiles, Mark Start And Exit, Exit Pad Size, the Dungeon Renderer used to draw the result, Generate On Start, Generate Async, and Saved Layout.
- **Advanced** (collapsed) — fine-tuning you rarely need: Min Split Size, Max Generation Attempts, Extra Leaf Count, Leaf Cluster Weight, Doorway Bias Depth, the bump/notch size ranges, Async Columns Per Frame, gizmos, and verbose logging.

After each build, a readout shows rooms, boss/treasure/shop counts, seed, and attempts used.

Corridor Width. 1 gives classic narrow corridors. Use 2 or 3 when your character has a physics collider close to one tile wide, so it does not scrape the walls. Doorways widen to match.

Start and exit markers. With **Mark Start And Exit** on, the start room is painted with the Start tile and a square Exit pad (the End tile, **Exit Pad Size** cells across) sits in the middle of the exit room, so both are easy to spot.

Layout Mode controls how rooms connect:

- **Single Line** — one path from start to end. Every middle room has exactly two doors; only the start and the exit are dead-ends.
- **Branches** — a tree. Rooms can fork off side paths, but there is never a circular route.
- **Loops** — branches plus a few extra corridors, so rooms can connect in circular paths (the default).

In every mode the exit/boss room stays a single-entrance dead-end.

Seeds. With **Use Random Seed** on, each Generate picks a new seed. Turn it off and set **Seed** to reproduce a specific dungeon exactly. The seed actually used is reported in the readout and in `LastEffectiveSeed`.

4. Tiles & themes

For each cell type (Floor, Wall, Corridor, Start, End, Boss, Treasure, Shop), tiles resolve in this order (Boss, Treasure and Shop fall back to the floor tile when a theme leaves them empty):

- 1 **Theme** — assign a `DungeonTheme` to the renderer's **Theme** slot (create one via **Assets → Create → Hybrid Dungeon Kit → Dungeon Theme**). A theme overrides the individual tile fields, so one swap re-skins the whole dungeon.
- 2 **Individual tile fields** on the renderer, when no theme is assigned.
- 3 **Placeholder colour tiles**, generated automatically when a tile is still missing. These are preview-only — assign real tiles or a theme for shipping. Toggle them with **Use Color Fallback When Unassigned** and recolour them on the renderer.

Sample themes. Run **Tools → Hybrid Dungeon Kit → Build Sample Theme** to generate two ready-made themes from the shipped tiles: **Dungeon** (pixel art) and **Colors** (flat colours), including treasure and shop tiles. It also wires a theme switcher into the scene (see below). Run it again after updating the kit to pick up newly added tiles.

Rule Tiles and animated tiles

Every tile slot accepts any `TileBase`, so **Rule Tiles**, **Animated Tiles** and **Random Tiles** from Unity's *2D Tilemap Extras* package work out of the box. Install it from **Window → Package Manager → Unity Registry → 2D Tilemap Extras**, create a tile via **Assets → Create → 2D → Tiles → Rule Tile**, and drop it into a theme slot.

- **Walls** are all drawn on the Wall tilemap, so a wall Rule Tile sees its neighbours and picks corners and edges correctly. Leave the Rule Tile's default collider (or set it to Grid) so walls stay solid.
- **Floors:** by default corridors are on their own tilemap, so a floor Rule Tile treats corridor openings as edges. If you want floors and corridors to blend, tick **Draw Corridors On Floor Tilemap** on the renderer and use the same Rule Tile (or a compatible one) for both.
- **Special floors** (Start, End, Boss, Treasure, Shop) sit on the Floor tilemap too. A Rule Tile that uses "This" neighbour rules sees them as different tiles; use "Not This" rules, or give them the same Rule Tile, if the seams bother you.
- **Wall tiles you make yourself** must have a collider. A plain Tile with **Collider Type = None** produces no collision; set it to **Grid**. The renderer warns in the Console if it finds one.

Switching themes

Add a **Dungeon Theme Switcher** component (on the generator object). Give it the renderer, the generator, and a list of themes. It re-skins the **current** dungeon without regenerating it — the layout stays, only the tiles change.

- In the editor: use its **Previous / Next** buttons, or the per-theme buttons, to preview each theme in edit mode.
- At runtime: call `NextTheme()`, `PreviousTheme()`, or `ApplyTheme(index)` from your UI buttons. The `On Theme Changed` `UnityEvent` fires on every switch.

5. Boss, treasure & shop rooms

Boss rooms

- **Boss Rooms Per Dungeon** — how many rooms become boss rooms. The exit room is always chosen first, then the largest remaining rooms. The start room is never a boss room; the value is clamped to the room count.
- **Boss Prefab** — spawned at the centre of each boss room (falls back to the theme's boss prefab).
- **Mark Boss Tiles** — paints boss-room floors with the Boss tile.

The exit/boss room is always the **farthest single-entrance dead-end** from the start: exactly one corridor leads into it, so the player is funnelled to the climax with no route around it. Read boss rooms back with `generator.BossRooms` and `generator.IsBossRoom(roomIndex)`.

Treasure and shop rooms

- **Treasure Rooms Per Dungeon** — rewards for exploring. Dead-end rooms are preferred, then the rooms deepest into the dungeon.
- **Shop Rooms Per Dungeon** — placed around the middle of the run, preferring rooms on the main route so players pass by naturally.
- **Mark Special Room Tiles** — paints their floors with the Treasure / Shop tiles.

Special rooms are never the start or a boss room. Bosses are chosen first, then treasure, then shops; each count is clamped to the rooms still available. Find them with `generator.GetRoomsOfType(DungeonRoomType.Treasure)` (or `.Shop`), and fill them with the Dungeon Spawner's **Treasure Rooms** / **Shop Rooms** filters. In the Scene view, treasure rooms are outlined orange and shops cyan.

6. Multi-level runs (optional)

Add a **Dungeon Run Controller** for floor-by-floor progression:

- **Number Of Levels** — floors in a run.
- **Base Room Count / Room Count Per Level** — room difficulty ramp.
- **Base Boss Rooms / Boss Rooms Per Level** — boss difficulty ramp.
- **Themes By Level** — optional per-floor theme (the last is reused if the list is shorter than the level count).
- **On Level Generated (int) / On Run Completed** — UnityEvents to hook your own logic (place the player, update UI, etc.).

Call `AdvanceLevel()` from your exit trigger to descend. The controller disables the generator's own **Generate On Start** so it stays in charge.

7. Rooms, spawning & events

Solid walls

The renderer adds a collider to the wall tilemap after each render, so walls are solid immediately — your player cannot walk through them. On the **Tilemap Dungeon Renderer**:

- **Add Wall Collision** — on by default. Turn off if you handle collision yourself.
- **Use Composite Collider** — merges the wall tiles into one smooth collider (recommended: fewer colliders, and characters do not snag on tile seams).

Your player needs a `Rigidbody2D` and a `Collider2D` to be stopped by them.

Room data

Every generated room is available as a `DungeonRoom`:

Member	Meaning
Index	Position in <code>generator.Rooms</code>
Rect	The room's rectangle, in grid cells
Type	Normal, Start, Boss, Treasure or Shop
IsExit	True for the dungeon's exit room
ConnectedRooms	Indices of rooms joined by a corridor
Doorways	Cells just inside the room where corridors meet
CenterCell / CenterWorld	The room's centre
IsDeadEnd	True when only one corridor leads in
DistanceFromStart	Rooms between the start and this one (0 = start). Handy for scaling difficulty

Look rooms up with `generator.Rooms`, `GetRoom(i)`, `GetRoomAtCell(cell)`, `GetRoomAtWorld(position)`, `StartRoom` and `ExitRoom`. Grid cells map 1:1 to world units, so cell (3, 5) is world position (3, 5).

Spawning enemies and loot

Add a **Dungeon Spawner**, point it at the generator, and add rules. Each rule picks prefabs, chooses which room types to fill, and how many to place:

- **Which rooms** — Normal / Boss / Start / Exit / Treasure / Shop, or *Dead End Rooms Only*.
- **How many** — Chance Per Room, plus a Min/Max count.
- **Placement** — Wall Margin and Door Clearance keep spawns off the walls and out of doorways.

The spawner refills automatically after each generation and clears the previous batch first. Typical setup: one rule for enemies (normal rooms, 1–3 each), one for chests (treasure rooms, exactly 1), one for a shopkeeper (shop rooms, exactly 1).

Reacting to the player's position

Add a **Dungeon Room Tracker**, point it at the generator and your player. It fires UnityEvents as the player moves:

- **On Room Entered / On Room Exited** — every transition.
- **On Room First Entered** — the first visit to each room, ideal for spawning an ambush or revealing a minimap once.

In code, use the `RoomEntered`, `RoomExited` and `RoomFirstEntered` C# events, or read `CurrentRoom`, `VisitedRoomCount` and `HasVisited(index)`.

```
void Start()
{
    tracker.RoomEntered += room =>
    {
        if (room.Type == DungeonRoomType.Boss)
            StartBossMusic();
    };
}
```

8. Minimap & fog of war

Minimap

Add a **Dungeon Minimap** (Add Component → Hybrid Dungeon Kit → Dungeon Minimap) and set its **Generator**. It needs no UI setup: after each generation it draws the dungeon into a small texture and shows it in a screen corner, with a marker for the player. The demo scene includes one.

- **Player** — what the marker follows. If empty, an object tagged `Player` is used.
- **Reveal** — *Whole Map*, or *Explored Only* (fog of war): rooms appear when the player enters them, and corridors are uncovered within **Reveal Radius** cells as the player walks.
- **Display** — corner, size, margin, opacity, player marker size, and colours for every room role.

To show it in your own Canvas instead, turn off **Draw On Screen** and assign the texture to a `RawImage` (each dungeon gets a new texture, so re-assign on rebuild):

```
minimap.TextureRebuilt += tex => rawImage.texture = tex;
```

Call `RevealAll()` for a map pickup, `RevealRoom(room)` / `RevealArea(rect)` for scripted reveals, and toggle `Visible` from your own input.

If you also use Fog Of War, drag it into the minimap's **Fog Of War** slot so the minimap shows exactly what the fog has uncovered.

Fog of war

Add a **Dungeon Fog Of War** (Add Component → Hybrid Dungeon Kit → Dungeon Fog Of War), set its **Generator**, and give it your **Player** (or tag the player `Player`; the sample `Player Spawner` does this for you). In Play mode the dungeon starts in darkness and is uncovered as the player explores. Fog is never shown in edit mode, and the fog layer is created for you on the dungeon's Grid.

- **Reveal Radius** — cells around the player that are in view.
- **Reveal Room On Enter** — light the whole room the moment the player steps in.
- **Remember Explored** — keep places you have seen dimly visible (the classic roguelike look). Off: they go fully dark again when out of view.
- **Unexplored / Explored Color** — the darkness colours; lower the alpha for a lighter fog.
- **Sorting Order** — keep it above the dungeon and your enemies and below your UI, so unseen enemies are hidden too.

From code: `IsExplored(cell)`, `IsVisible(cell)`, `RevealAll()`, `Rebuild()`, and the `CellsExplored` event. The fog reveals by distance and does not do line of sight through walls.

9. Saving, loading & baking

Save and load layouts (JSON)

Save Layout... writes the current dungeon to a small JSON file (a 300×300 dungeon is about 7 KB): the tile grid, every room and its role, and the corridors. **Load Layout...** brings it back exactly, even if you have since changed the generator's settings. Loading behaves like a normal generation: bosses are respawned, the tiles are drawn, and `OnGenerated` fires, so the spawner, minimap, fog and camera all update.

To start a scene with a fixed layout, save the JSON inside your Assets folder and drag it into the generator's **Saved Layout** slot. With Generate On Start on, that layout is loaded instead of a random one.

From code (e.g. save games):

```
string path = Path.Combine(Application.persistentDataPath, "floor3.json");
generator.SaveLayoutToFile(path);
// later
generator.LoadLayoutFromFile(path);

// or keep the JSON yourself
string json = generator.ExportLayout().ToJson();
generator.LoadLayout(DungeonLayoutData.FromJson(json));
```

Invalid or corrupt data is rejected with a Console warning and leaves the current dungeon unchanged. Tip: saving just the **seed** also recreates a dungeon, but only while your settings and kit version stay the same; a saved layout is exact forever.

Baking to a prefab

Found a layout you love? Click **Bake to Prefab** on the generator (or **Tools → Hybrid Dungeon Kit → Bake Selected Dungeon to Prefab**) and choose where to save. The prefab contains the Grid with its tilemaps and wall colliders, plus spawned bosses and Dungeon Spawner objects — but no generator, so it never changes. Use it as a hand-tuned fixed level, a tutorial floor, or a starting point for manual editing.

Baking needs real tile assets: the placeholder colour tiles exist only in memory. If you see a warning, assign a theme (**Build Sample Theme** gives you two), Generate, then bake. Bake in Play mode to include the Dungeon Spawner's objects.

10. Scripting API

```
using HybridDungeonKit;

public class MyGame : MonoBehaviour
{
    public HybridDungeonGenerator generator;

    void Start()
    {
        generator.OnGenerated += g =>
        {
            Vector3 spawn = g.GetStartWorldPosition(); // place your player here
            Vector3 exit  = g.GetEndWorldPosition();
            int[,] grid    = g.GetGrid();                // raw tile codes (DungeonTile.*)
        };

        generator.targetRoomCount = 12;
        generator.bossRoomsPerDungeon = 2;
        generator.Generate();
    }
}
```

Key members

- Build/clear: `Generate()`, `GenerateAsync()`, `ClearGenerated()`, `IsGenerating`
- Save/load: `ExportLayout()`, `LoadLayout(data)`, `SaveLayoutToFile(path)`, `LoadLayoutFromFile(path)`; `DungeonLayoutData.ToJson()` / `FromJson(json)`
- Grid: `GetGrid()`, `GetTile(x, y)`
- Rooms: `GetRoomRects()`, `GetRoomCount()`, `GetStartRoom()`, `GetEndRoom()`, `RoomCenter(rect)`
- World positions: `GetStartWorldPosition()`, `GetEndWorldPosition()`
- Bosses: `BossRooms`, `IsBossRoom(i)`, `GetBossRoomCount()`
- Rooms: `Rooms`, `GetRoom(i)`, `GetRoomAtCell(cell)`, `GetRoomAtWorld(pos)`, `StartRoom`, `ExitRoom`, `GetRoomsOfType(type)`, `TryGetRandomWalkableCell(room, out cell)`, `IsWalkable(x, y)`
- Result info: `LastGenerationSucceeded`, `LastEffectiveSeed`, `LastAttemptsUsed`
- Events: `OnGenerationStarted`, `OnGenerationProgress (0-1, async only)`, `OnGenerated` (raised after every successful build)

Tile codes live in `DungeonTile`: `Empty`, `Floor`, `Wall`, `Corridor`, `Start`, `End`, `Boss`, `Treasure`, `Shop`. `DungeonTile.IsWalkable(code)` tells you whether a code can be walked on.

11. Samples

The **Samples** folder contains the example components used by the playable demo. Copy and adapt them; none are required by the kit.

- **Dungeon Player Spawner** — put it on the generator; it places a player at the start room and an exit trigger on the exit pad each time a dungeon is built. With no prefab it creates a walkable placeholder player tagged `Player`. Your own player prefab must be tagged `Player` for the exit trigger to react.
- **Dungeon Exit Trigger** — drop it on a trigger collider; it advances the run (or regenerates) when the player enters.
- **Dungeon Player Controller** — a minimal top-down Rigidbody2D mover.
- **Dungeon Camera Follow** — a smooth orthographic follow camera.
- **Dungeon Demo HUD** — the demo's help panel, room readout and hotkeys.
- **Dungeon Demo Input** — works with the new Input System, the legacy Input Manager, or both, whichever your project uses.
- **Create Playable Demo Scene** (Tools menu) — rebuilds the demo scene and its sample Enemy / Chest / Shopkeeper prefabs.

If you don't need them, you can delete the Samples and Examples folders; the rest of the kit does not depend on them.

12. Performance & async generation

Generation cost scales with grid area and room count. The default 300×300 grid with ~10 rooms builds in a few milliseconds; tiles are drawn with batched `SetTiles` calls. Increase **Max Generation Attempts** if a small grid with many large rooms occasionally fails.

For very large grids (500×500+), use async generation so the game never freezes: tick **Generate Async** to build on Start, or call it from code. Layout attempts run one per frame and the tiles are drawn **Async Columns Per Frame** columns at a time. The same seed gives the same dungeon either way.

```
generator.OnGenerationStarted += g => loadingScreen.SetActive(true);
generator.OnGenerationProgress += p => loadingBar.value = p;    // 0..1
generator.OnGenerated += g => loadingScreen.SetActive(false);
generator.GenerateAsync();
```

`IsGenerating` is true while a build is in progress; a second `Generate` call during that time is ignored with a warning. In edit mode, `GenerateAsync()` simply builds immediately.

13. FAQ & troubleshooting

Nothing is drawn after Generate. Make sure the generator has a **Dungeon Renderer** with three tilemaps assigned — click **Initialize Scene**. The renderer also warns in the Console if its tilemaps are missing.

"Generation failed after N attempts." The grid is too small for the requested rooms. Lower Target Room Count or Min Room Size, raise the grid Width/Height, or increase Max Generation Attempts.

Camera doesn't show the whole dungeon. Add a **Dungeon Camera Framer** to your camera and set its Generator — it auto-fits the view after each generation (the demo scene already includes it), or click its **Frame Now** button.

Colours instead of art. That's the placeholder fallback. Assign your tiles or a **DungeonTheme** to the renderer, or turn off **Use Color Fallback When Unassigned**.

"Cannot bake: placeholder colour tiles." Assign a **Dungeon Theme** to the renderer, Generate, then bake again (see 9).

My player walks through walls. The player needs a **Rigidbody2D** and a **Collider2D**, and the wall tile needs **Collider Type = Grid**. Sample themes built by earlier versions of the kit set it to None; run **Build Sample Theme** again to fix them.

Fog covers everything / nothing. Fog needs a player: assign one or tag it **Player**. It only appears in Play mode.

My character gets stuck in corridors. Raise **Corridor Width** to 2 or 3, and keep **Use Composite Collider** on so walls have no seams.

Same dungeon every time. Turn on **Use Random Seed**, or change **Seed**.

Boss room has more than one entrance. The primary boss (the exit) is always a single-entrance dead-end. Extra boss rooms (when Boss Rooms Per Dungeon > 1) are the largest rooms and may have several entrances.

14. Support

For questions or issues, contact: **abdulrehman60dev@gmail.com** Please include your Unity version and render pipeline.